International Journal of Software Engineering
and Knowledge Engineering
Vol. 26, No. 5 (2016) 733–768

DOI: 10.1142/S0218194016500248

World Scientific
www.worldscientific.com

Regression Test Cases Prioritization Using Clustering and Code Change Relevance

Ramzi A. Haraty*, Nashat Mansour†, Lama Moukahal‡
and Iman Khalil§

Department of Computer Science and Mathematics
Lebanese American University Beirut, Lebanon
**rharaty@lau.edu.lb*
†*nmansour@lau.edu.lb*
‡*lama.moukahal@lau.edu.lb*
§*iman.khalil@lau.edu.lb*

Received 20 June 2014
Revised 21 July 2014
Accepted 19 July 2015

Regression testing is important for maintaining software quality. However, the cost of regression testing is relatively high. Test case prioritization is one way to reduce this cost. Test case prioritization techniques sort test cases for regression testing based on their importance. In this paper, we design and implement a test case prioritization method based on the location of a change. The method consists of three steps: (1) clustering test cases, (2) prioritizing the clusters with respect to the relevance of the clusters to a code change, and (3) test case prioritization within each cluster based on metrics. We propose a metric for measuring test case importance based on Requirement Complexity, Code Complexity, and Code Coverage. To evaluate our method, we apply it on a launch interceptor problem program, and measure the inclusiveness and precision for clusters of test cases with respect to code change in specific test cases. Our results show that our proposed change-based prioritization method increases the likelihood of executing more relevant test cases earlier.

Keywords: Clustering; regression testing; software testing; test case prioritization.

1. Introduction

Regression testing is a software testing method that searches for software bugs after certain modifications take place. Regression testing brings many advantages where it assures the software quality, and guarantees that no additional errors are caused by modifying the software. This is accomplished by selecting a reduced subset of test cases of the initial test cases. Initially, a set T of N test case is saved after the initial

*Corresponding author.

software development. After a software modification, regression testing requires a subset of test cases (R) to be selected for rerunning the test again on the modified software. Rerunning the subset of test cases (R) should guarantee that the modification did not adversely affect the software. However, re-executing test cases is a resource-consuming activity especially for software with a high number of test cases.

For example, Carlson *et al.* [1] performed regression testing on Microsoft Dynamics Ax software. The results showed that executing test cases requires a significant number of days; moreover, the experiment showed that analyzing the results took more number of days than executing the test cases themselves.

Rothermel *et al.* [2] stated that there are three types of techniques to reduce the cost of regression testing: regression test selection techniques, test suite minimization techniques, and test case prioritization techniques. Regression test selection techniques aim to select a proper subset of the initial test suite based on the program information. Test suite minimization techniques reduce the regression testing cost by extracting a minimal subset that maintains the coverage of the initial test suite based on a certain test criteria.

Test case prioritization techniques are methods for minimizing the regression testing cost. These techniques allow test engineers to prioritize their test cases according to their importance; during the regression testing activity, the important test cases run at an earlier stage. Test cases should be prioritized or sorted according to their influence by means of a measure metric. As an example for a measure metric, a metric may reflect the speed of test cases in detecting an error, or it could be the code coverage at the fastest possible rate. Test case prioritization techniques do not ignore test cases' details as regression test selection and test suite minimization do, but rather their techniques delay the execution of test cases that are not effectively related to the change when the resources permit this action.

A small number of clustering-based test case prioritization papers have appeared in the literature. Carlson *et al.* [1] introduce and implement a test case prioritization technique that utilizes a clustering approach in order to improve the number of detected faults when only a certain number of test cases are executed. This technique is composed of two steps: clustering test cases, and prioritization test cases within the cluster. Lenz *et al.* [3] present an approach that ensures a program is tested sufficiently through measuring its coverage. They propose a fault-based approach with a machine learning technique.

Simons and Parsio [4] present an optimization technique for regression testing by adopting a prioritization technique that introduces changes to a failure pursuit sampling technique in order to focus on regression tests. Xian [5] presents a dynamic test case prioritization technique based on the design information of the test suite, namely, the design purposes of the test cases and the similarity of purposes. Kayes [6] presents a new metric to measure the fault dependency detection rate and proposes a test cases prioritization algorithm. The algorithm first determines the number of faults dependent (NFD) on each fault based on a fault dependency matrix. Then it computes the total dependency count (TDC) of each test case. The TDC of a test

case is simply the summation of NFD of faults that are first exposed in the test case. Using the value of TDC, the algorithm then sorts the test cases in descending order. The ART algorithm, proposed by Zhang *et al.* [7], aims to use test case coverage information to the extreme. This algorithm employs a procedure, called Generate, to iteratively build a test case candidate list. As long as a random test case can increase the program coverage, the test case will be added to the candidate list. To select a candidate test case, the algorithm invokes another procedure that calculates the distance between two test cases based on the coverage constructs, and it returns the index of the selected farthest test case away from the prioritized set that has been covered so far.

Walcott *et al.* [8] present a time aware technique that uses genetic algorithms to prioritize test cases that will run within a constrained execution environment. Similarly Zhang *et al.* [9] present a time aware technique using integer linear programming. The authors performed experimental results to compare their technique with four traditional techniques for test case prioritization. When the time budget is tight, the proposed method performs superior to other methods.

Regression testing is an expensive tool as stated by Kim and Porter [10]. To reduce the expenses of regression testing, the authors propose a new technique to prioritize test cases based on the historical execution of data. In other words, the logged history of test cases execution data will specify the set of test cases that need to be first executed in the regression testing activity.

Mirarab *et al.* [11] propose a novel approach in order to select a subset of test cases. The approach works by forming an integer linear programming problem using different coverage criteria. In addition, the approach takes into consideration the test cases that are close to the optimal solution. These close to optimal solution points are found using constraint relaxation. Later, the selected subtest cases are prioritized based on a greedy algorithm. Conversely, Pravin *et al.* [12] prioritized test cases using a fault detection algorithm.

Another study conducted by Sherrif *et al.* [19] clarifies that any change made to a tested system can result in new faults that require repeating the testing phase of the system. The authors proposed a new methodology that determines the effect of a code change on the system. Their methodology depends on the historical data, where the test cases are clustered based on their historical changes. Beszedes *et al.* [20] discussed in their paper the cost of regression testing and stressed on the importance of prioritizing the test cases. In their approach, they used the code coverage in order to prioritize the test cases; thus, only the test cases that cover some parts of the change are considered to be affected by the change.

In this paper, we propose a test case prioritization method based on both clustering and specific change location. Clustering is used for its well-known power in grouping objects together that share similar structure. Thus, clustering test cases based on Requirement Complexity, Code Complexity, and Code Coverage helps in improving test case prioritization techniques. The proposed method clusters the test cases based on test case similarities. Then, it prioritizes clusters based on the

specific change location. Finally, the algorithm prioritizes test cases within a cluster based on the importance of a proposed metric. To evaluate our work, we performed an experimental study over a Lunch Interceptor System. The most relevant to our work reported herein is that of Carlson *et al.* [1], which introduced a technique to prioritize test cases based on a clustering approach. However, Carlson *et al.* did not account for the location where the code change took place and its implications. They did not prioritize clusters, but test cases within the clusters. Other papers randomly select test cases from clusters [13–15, 17]. Our contribution is that we prioritize clusters based on the location of the change and use a new metric to measure the importance of a test case. In summary, the important contributions of this paper are as follows:

- Select reduced subset of test cases from the initial test cases by clustering test cases (Sec. 3.1).
- Minimize the regression testing cost by prioritizing test cases (Sec. 3.3). This is done by sorting the test cases within each cluster based on their importance. Three factors affect the test case importance:
  - Code Coverage
  - Code Complexity
  - User Requirement Complexity.
- Generate the test case prioritized list, where our method will pick the sorted test cases from the cluster with the highest code change relevance ratio. Then, we move to the next cluster and pick the test cases until all clusters are covered (Sec. 3.3).
- The method is tested on the Launch Interceptor Program (LIP), and the results show that the method prioritizes test cases effectively.

The rest of the paper is organized as follows. Section 2 describes the problem. Section 3 discusses our proposed algorithm's design and implementation. Section 4 presents the experimental results. Section 5 concludes the paper and presents future work.

2. Problem Description

Test case prioritization techniques sort test cases based on certain criterion to be executed in the regression testing activity. The goal of these techniques is to increase the probability of meeting some goals during regression testing at the time the test cases are executed in the prioritized order [15]. In our problem, we will focus on the objective that testers want to increase the likelihood of detecting faults related to a specific code change earlier; that is, without having to run all test cases. We also aim to fulfill the second level of prioritization objective: test cases with the highest code coverage, and those that are more critical to the client, and have a higher complex code will to be executed with higher prioritization.

2.1. *Problem definition*

Whenever an update occurs in one of the methods of the program in question, our aim will be to prioritize test cases according to their influence with the change that occurred. A change is any modification that is applied to any method of the program; for example, a change in the Lunch Interceptor Program can be a change in the calculation of the velocity of the missile, or a change in the trajectory function of the missile.

The program in question is represented by a graph where the nodes of this graph are the methods of the program. For every set of test case, PT represents the set of all possible orderings of T. Moreover, f is a function that when applied to any ordering (any code change), it will prioritize test cases in the idle way. The resulting test cases will be prioritized with an attempt to increase their effectiveness.

The test case prioritization problem is defined as the following given:

(1) $T = \{t_1, t_2, t_3, \ldots, t_N\}$ is the set of the program's N test cases, PT is the set of permutations of T; PT represents all the possible orderings of T.
(2) Graph G is the segment-sequence graph of the program flow graph. G is a directed graph. G's vertex set is the non-decision segments set (S) of the problem's flow graph. The edge set of G represents the ability of a test case to traverse from one segment to another in the flow graph.
(3) Graph G' is a directed graph. It represents the decision segments of the program's flow graph. G' vertices are the decision segments (D) of the problem flow graph. The edge set of G' represents the ability of a test case to traverse a decision segment of the flow graph.
(4) si is a node that belongs to G where the code change took place.
(5) ti is a single test case. $\forall\ ti \in T$, ti is the set of traversed nodes in G.

The problem is to find $T' \in \mathrm{PT}$ such that $(\forall T'')\ T'' \in \mathrm{PT}$ and $T'' \neq T'$, $f(T') > f(tT'')$, where function f is a quantitative measure that satisfies the goal of increasing the probability of satisfying the objective mentioned earlier in this section.

2.2. *Data structures*

We model a program using a segment–sequence graph G with a set of vertices/segments, $S = \{s1, s2, \ldots, s_m\}$, where m is the total number of non-decision segments in the program's flow graph. A segment refers to a sequence of simple statements or method name. Based on the model, we build a data structure, X, that represents the paths of test cases through non-decision segments of the flow graph. Each path is defined by the segments that the test case traverses.

Given $D = \{d1, d2, d3, \ldots\}$, where di is a decision segment in the program's flow graph. Based on graph G', we build a data structure, X', that represents the paths of test cases through decision segments. Each path is defined by the segments that the test case traverses.

Also, we construct a matrix, M, that stores all inter-cluster distances, as defined in the following section.

3. Clustering and Change-Based Prioritization Method

In this section, we present our proposed clustering change-based test case prioritization algorithm. Figure 1 describes the proposed algorithm's phases and steps, while Fig. 2 shows a process flow diagram for the entire procedure. Our method is composed of three main phases. The first phase — test case clustering — groups similar test cases in a cluster. The second phase prioritizes clusters according to the specific segment change relevance. The third phase prioritizes test cases within each cluster.

3.1. *Clustering test cases*

Define an initial set of clusters, $C = \{C_1, C_2, \ldots, C_i, \ldots, C_N\}$, where N is the initial count of total clusters and C_i represents the ith cluster. A cluster C_i represents set of

```
Algorithm:  ChangeBasedPrioritization(T, si, G, G', FCN)
Input:      T    = Set of test cases.
            si   = Segment location where the change
                   took place.
            G    = Program segment sequence.
            G'   = Program decision nodes sequence.
            FCN  = Designated number of clusters.
Output:     T'   = Prioritized test cases set for
                   regression testing.
Description:Sort T to fill the problem objectives.
Phase I – Clustering Test Cases
        1.  InitializeClusterDistanceMatrix(T)
        2.  UpdateMatrixDistance
        3.  While(CurrentNumberofClusters > FCN)
             Begin while
                  MergeClosestClusters
                  UpdateMatrixDistance
             End while
Phase II – Change-based Cluster Prioritization
        4.  GetTestCasesRelevanceRatio
        5.  CalculateClustersRelevanceRatio(si)
        6.  PriortizeClusters
Phase III – Test Cases Prioritization
        7.  PrioritizeTestCasesWithinCluster
        8.  T' = PrioritizedTestCasesList
```

Fig. 1. The change-based prioritization algorithm.

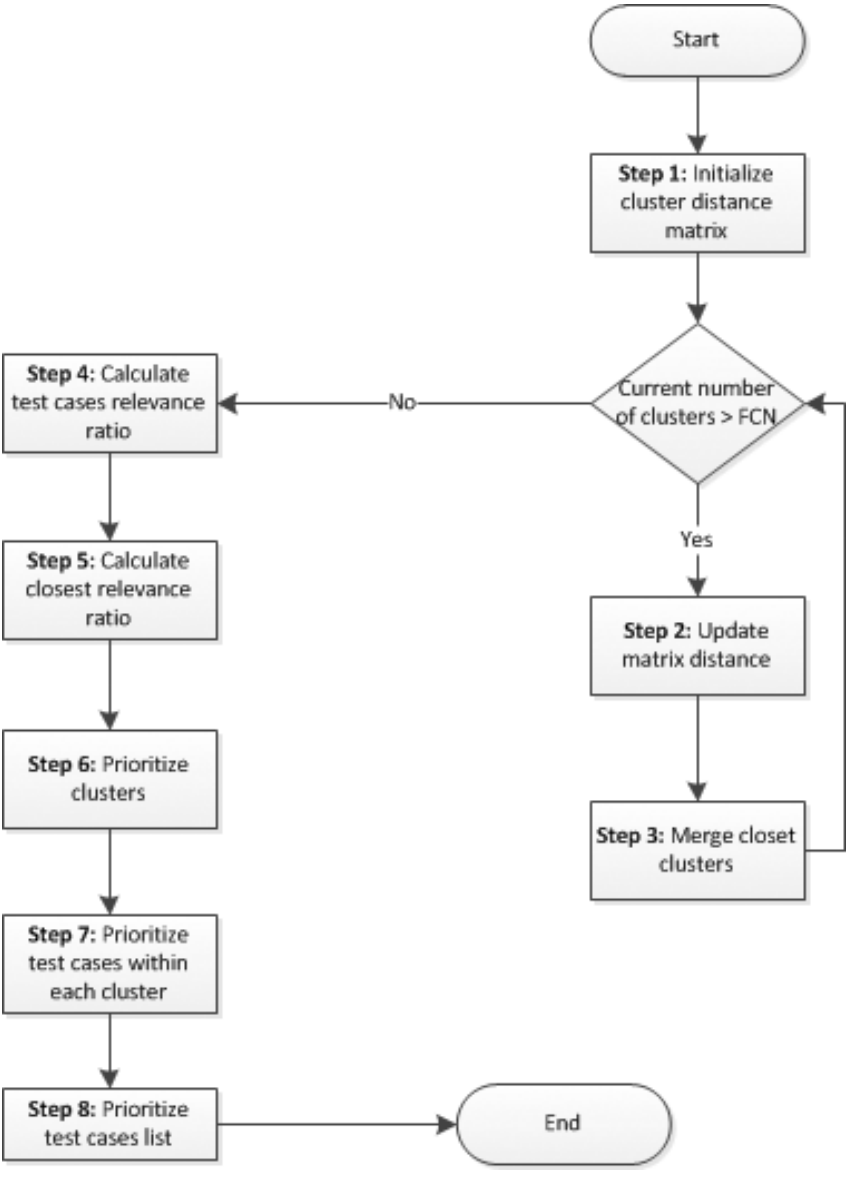

Fig. 2. Flow diagram for the change-based prioritization algorithm.

test cases. Cluster C_i is represented as vector of test cases: $C_i = \{t_1, t_2, t_3, \ldots, t_k\}$, where k represents the count of test cases within cluster i. Initially, the ith cluster, C_i, contains only the wth test case (t_w). The wth test case (t_w) represents a set of traversed segments: $t_w = \{s_1, s_2, \ldots, s_m\}$ where m is the number of total segments traversed by the whole problem. S_z is a 0–1 value that represents whether the designated test case traverses the zth segment or not.

The final number of clusters (FCNs) is a predefined parameter that should be in the interval of (2, 10). FCN is comparative to the number of test cases. The more the test cases exist, the more the FCN increases. However, FCN is proportional to the location of the segment in the workflow where the change took place. If the location is near the beginning, the final number of clusters increases. FCN eventually decreases while the change location of the change is in the neighborhood of the end of the program. On the other hand, FCN is inversely proportional to the code complexity. More test cases should be included in a cluster to ensure sufficient testing.

The algorithm's first phase is to cluster test cases based on distance similarity. Using an Agglomerative hierarchal clustering method, test cases will be clustered into a predefined number of clusters. The test cases are clustered according to the following steps:

Step 1 — Initialize Cluster Distance Matrix
Initially assign each test case to a cluster; i.e. the initial number of clusters is equal to the number of test cases.

Step 2 — Update Matrix Distance
This step finds the similarity measure for all pair-wise clusters. In our approach, the similarity measure of two clusters is based on the distance between them. Given two clusters C_i and C_j, the distance between cluster i and cluster j, $d(C_i, C_j)$, is the average of distances between all test cases in C_i and test cases in C_j. The distance, $d(C_i, C_j)$, between cluster i and cluster j is obtained as follows:

$$d(C_i, C_j) = \frac{\sum_{k=1}^{n} \sum_{l=1}^{n'} d(t_k, t_l)}{n + n'}, \tag{1}$$

where n is the total number of test cases in cluster C_i and n' is the total number of test cases in C_j. $d(t_k, t_l)$ is the distance between test case k and test case l. Keeping in mind that test cases are vectors of 0–1 (the value represents the traversed segments), test cases vectors are of fixed length. Distance functions a have huge effect on clustering. Singla and Karambir [18] showed that the Euclidean distance function takes less computational time than the Manhattan distance function. Accordingly, the Euclidean distance function is used to calculate $d(t_k, t_l)$ in order to improve the computational time needed to calculate the distance. The Euclidean distance function is used to obtain $d(t_k, t_l)$ as follows:

$$d(t_k, t_l) = \sqrt{\sum_{b=1}^{m} (x_{kb} - x_{lb})^2} \tag{2}$$

where m is the total number of defined methods in the problem. x_{jb} as represented in the matrix M, is used to determine whether test case j traverses segment b or not. x_{jb} is set to 0 if the test case does not traverse the bth segment; otherwise, it is set to 1.

Step 3 — Merge Clusters and Update Matrix
We obtain the shortest distance value in the distances matrix M. Then, each cluster pair having the minimum distance value will be merged to a single cluster. Next, we rebuild the distance matrix by updating the distance between the merged clusters and the other clusters (refer to step 2 to rebuild the distance matrix (M)). We repeat step 3 until the current number of clusters is reduced to the designated number of clusters.

3.2. *Change-based cluster prioritization*

Step 4 — Calculate Test Case Relevance Ratio
In our approach, the program segment sequence is represented by a directed graph G. G's vertex set is represented by $S = \{s_1, s_2, s_3, \ldots, s_m\}$, where s_i represents a node of the flow graph problem. The graph's edge set represents the possibility of traversing a segment from another (or dependency of a segment on another). Figure 3 shows a segment sequence example where segment s_2 can be traversed after s_1. The set of edges, $E = \{(s_1, s_2), (s_2, s_4), (s_4, s_5), (s_1, s_3), (s_3, s_4)\}$.

Our method focuses on prioritizing test cases that reveal defects caused by a certain change. To do so, we count the number of each test case segments that can be affected by the changed segment so we can calculate the test case relevance ratio. The relevance ratio is the value to measure the strength of test case relationship with the specific change. A test case may not traverse the changed segment itself, but it may traverse segments that could be affected by the change. In other words, a test case may traverse segments that have a path to segments included in the path traversal changed segment. The example in Fig. 3 shows that a change in segment s_2 will affect the test case (t) of path $\{s_1, s_3, s_4, s_5\}$ due to the fact that s_4 and s_5 may be affected by s_2. (t) has a relevance ratio equal to 2.

Step 5 — Calculate Clusters Relevance Ratio
The cluster relevance ratio is the average of relevance ratios of test cases that belong to this cluster.

Step 6 — Prioritize Clusters
Having calculated the clusters' relevance ratio, we sort the clusters based on their relevance ratio in descending order. The cluster that has the highest ratio is the most related cluster to the significant change. As a result, a tester should start testing the cluster with the highest ratio.

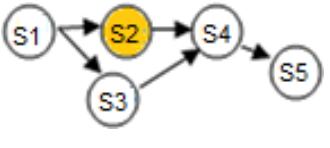

Fig. 3. A segment sequence graph.

3.3. *Test case prioritization*

Step 7 — Prioritize Test Cases within a Cluster
Having created and prioritized clusters, now we will prioritize test cases within each cluster. To do so, we have to sort test cases within each cluster based on their importance. Three factors affect the test case importance: Code Coverage (CCg), Code Complexity (CCy), and user Requirement Complexity (RC). These three factors are important in our study, since they identify the impact of a method in the software.

CCg is based on ranking test cases based on the segments coverage of a test case. When the number of segments that are executed or covered by a test case is high, then this test case has a significant effect; and thus, it is imperative that the test run earlier in the test cycle. CCy directly reflects the effort the development team made in order to develop a function. The more effort, the more complex the method is, nevertheless, complexity makes the method more error prone. Thus, code complexity is important to identify the methods that need to be treated with higher priority than other methods since errors might be more common in such methods. Software systems are prepared based on the user requirement. However, every user requirement has its own importance; some are critical, some are essentials, while others are less important to the user. RC helps in identifying the methods that contain features that are more frequently accessed by users and/or error-prone than other methods. Knowing that such information has huge impact on test prioritization since they reflect the importance of the method on the entire system.

All these factors reflect information about the test cases/methods, e.g. their complexity, their coverage, and their requirement. Thus, knowing the methods' characteristics will allow us to identify sensitive methods that might get affected drastically by any change. This will lead to prioritizing sensitive test cases at an early stage where a large number of faults will be detected directly. For example, the CCg factor will allow us to identify test cases that cover a lot of the code and as a result, executing them first means detecting more faults at an early stage. On the other hand, the RC helps in providing higher priority to the test cases that contain methods that are highly used. CCy can help in identifying test cases that contain complex methods enabling us to rank error prone test cases as significantly important.

The CCg metric is calculated by counting the number of segments traversed by a single test case. The CCy metric is determined by the number of decision nodes traversed by the test case t_i and the sum of statements since both contribute to the testing complexity. Referring to data structure X', x'_{ij} is used to determine whether test case t_i traverses decision node d_j or not. X_{ij} is set to 0 if the test case does not traverse d_j; otherwise it is set to 1. Assuming that decision nodes are 5 times more significant for test than simple statements, CCy is determined as follows:

$$\text{CCy} = \sum_{\text{path}} (\text{Statment Complexity}) + 5 * \sum_{\text{path (Decision nodes)}} . \tag{3}$$

RC is a user-defined value for each segment. The values of this metric range between 1 and 5. Value 1 corresponds to the least significant requirements such as display some non-critical information. Value 5 corresponds to the most significant ranked method, such as update customers account balance. Test case requirement complexity is the sum of methods requirement complexity the test case traverses. Therefore, the total test case priority (TP) is given by:

$$\mathrm{TP} = \alpha(\mathrm{CCg}) + \beta(\mathrm{CCy}) + \gamma(\mathrm{RC}), \tag{4}$$

where α, β, and γ are user defined integer values. α, β, γ are input parameters that depend on the problem's nature.

Step 8 — Create Prioritized Test Cases List
To generate the test case prioritized list, our method will pick the sorted test cases from the cluster with the highest code change relevance ratio. Then, we move to the next cluster and pick the test cases as well. We keep on choosing test cases of each cluster until all clusters are covered. The first test case in the list contains the most relevant segments to the change, and it is the most significant test case in terms of CCg, CCy, and RC. The tester starts picking test cases from the test cases prioritized list until all test cases are covered or the time/number of executed test cases reaches the limit.

4. Experimental Results

In this section, we evaluate our proposed method in improving the likelihood of detecting faults related to a specific code change earlier with wider CCg. Thus, we will perform an experimental study using the flow graph presented in Fig. 4 that is related to Launch Interceptor Program (LIP). The LIP is a realistic anti-missile system which was designed and implemented in the late 1980s. The program uses input data that represent radar reflections and computes a set of conditions based on some inter-relationships among these points. The LIP program was used as an experimental system for research conducted at Syracuse University on software reliability. The flow graph of Fig. 4 represents the methods of the LIP program.

We implemented the proposed algorithm by developing an application using C#. The application takes as an input the methods of the program and the segment in which the code change took place. The application is capable of automatically building the clusters and running the algorithm to prioritize the test cases. The code of the application is included in Appendix A.

The graph of Fig. 4 consists of 40 methods of the LIP program. A test case set (T) of 48 test cases were derived with the objective of all-statement testing, in addition several randomly-selected test cases are used as an input for the implemented method.

Referring to the algorithm presented in Fig. 2, the first step is to initialize cluster matrix M. Initially, we have 48 clusters since each test case is presented by a cluster. In the second step, the matrix distance is calculated. A sample of 20 test cases is

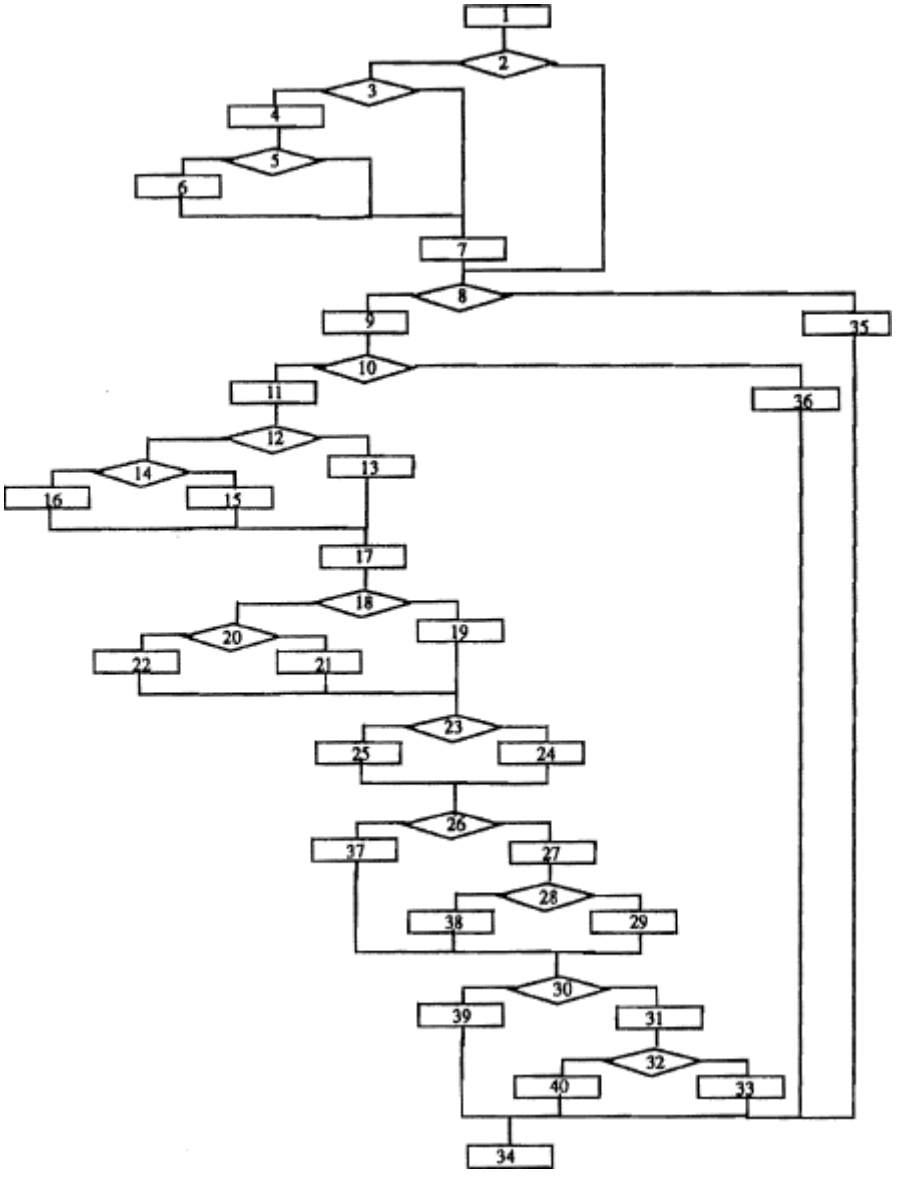

Fig. 4. LIP flow graph.

taken to clarify the proposed algorithm's steps. As shown in Table 1, each test case is considered to be a cluster and the matrix distance is calculated. The next step is to merge the clusters based on the minimum distance. Table 1 shows that the minimum distance is "1.4" — this leads to the merge of clusters 1 with cluster 4, and cluster 5 with cluster 20. This process is repeated until we reach a specific number of clusters that is specified by the need of the user.

Table 1. Initial clusters.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21	C22
C1	0	2	2.83	1.41	4.58	3.16	2.65	3.61	2.24	3.87	3.46	3.46	2	3	3.74	3.87	2	3.61	4.24	4.58	2.24	3.61
C2	2	0	2.45	2.45	4.58	2.83	3	3.32	3	3.87	3.16	3.74	2.83	2.24	3.74	3.87	2.83	3	4	4.58	2.65	3
C3	2.83	2.45	0	3.16	4.36	3.16	3	3.61	3.61	3.87	3.46	4	3.46	3.32	3.74	3.61	3.46	3.87	4.47	4.36	3.32	3.61
C4	1.41	2.45	3.16	0	4.36	2.83	3	3.32	2.65	3.61	3.74	3.16	2.45	3.32	3.74	4.12	2.45	3.87	4	4.36	2.65	3.32
C5	4.58	4.58	4.36	4.36	0	3.61	4	3.46	4.24	2.83	3.61	3.32	4.36	4.24	3	2.83	4.36	4	3	1.41	4.24	4
C6	3.16	2.83	3.16	2.83	3.61	0	3	3	3.61	2.65	2.45	2.45	3.74	3.32	2.83	3.32	3.74	3.87	3.74	3.61	3.61	3.32
C7	2.65	3	3	3	4	3	0	2.83	3.16	3.16	3.32	3.61	3.32	3.46	3	3.16	3.32	4	3.61	4	3.46	3.74
C8	3.61	3.32	3.61	3.32	3.46	3	2.83	0	4	3.16	3.87	3.61	4.12	3.74	3.32	3.16	4.12	4.24	3.32	3.46	4.24	3.74
C9	2.24	3	3.61	2.65	4.24	3.61	3.16	4	0	3.16	2.65	2.65	2.24	2.83	3.61	3.74	3	2.83	3.61	4.24	3.16	4
C10	3.87	3.87	3.87	3.61	2.83	2.65	3.16	3.16	3.16	0	2.24	1.73	3.87	3.74	2.24	2.83	4.36	3.74	2.65	2.83	4.47	4.24
C11	3.46	3.16	3.46	3.74	3.61	2.45	3.32	3.87	2.65	2.24	0	2	3.46	3	2.83	3	4	3	3.46	3.61	3.87	4.12
C12	3.46	3.74	4	3.16	3.32	2.45	3.61	3.61	2.65	1.73	2	0	3.46	3.61	2.83	3.32	4	3.61	3.16	3.32	4.12	4.12
C13	2	2.83	3.46	2.45	4.36	3.74	3.32	4.12	2.24	3.87	3.46	3.46	0	2.24	3.16	3.32	2.83	3.61	4.24	4.36	3	4.12
C14	3	2.24	3.32	3.32	4.24	3.32	3.46	3.74	2.83	3.74	3	3.61	2.24	0	3	3.16	3.61	2.83	3.87	4.24	3.46	3.46
C15	3.74	3.74	3.74	3.74	3	2.83	3	3.32	3.61	2.24	2.83	2.83	3.16	3	0	1.73	4.24	4.12	3.46	3	4.36	4.36
C16	3.87	3.87	3.61	4.12	2.83	3.32	3.16	3.16	3.74	2.83	3	3.32	3.32	3.16	1.73	0	4.36	4.24	3.87	2.83	4.47	4.69
C17	2	2.83	3.46	2.45	4.36	3.74	3.32	4.12	3	4.36	4	4	2.83	3.61	4.24	4.36	0	3	3.74	4.12	1.73	3.32
C18	3.61	3	3.87	3.87	4	3.87	4	4.24	2.83	3.74	3	3.61	3.61	2.83	4.12	4.24	3	0	2.65	3.74	3.16	3.16
C19	4.24	4	4.47	4	3	3.74	3.61	3.32	3.61	2.65	3.46	3.16	4.24	3.87	3.46	3.87	3.74	2.65	0	2.65	4.12	3.61
C20	4.58	4.58	4.36	4.36	1.41	3.61	4	3.46	4.24	2.83	3.61	3.32	4.36	4.24	3	2.83	4.12	3.74	2.65	0	4.47	4.24
C21	2.24	2.65	3.32	2.65	4.24	3.61	3.46	4.24	3.16	4.47	3.87	4.12	3	3.46	4.36	4.47	1.73	3.16	4.12	4.47	0	2.83
C22	3.61	3	3.61	3.32	4	3.32	3.74	3.74	4	4.24	4.12	4.12	4.12	3.46	4.36	4.69	3.32	3.16	3.61	4.24	2.83	0

For the example shown in Table 1, the final round is presented in Table 2.

After this step is completed, we start preparing for prioritizing test cases. Step 4 calculates the relevance ratio for each test case. In other words, in this step we calculate the strength of the relationship between the test cases and the segment that happened to change. In our example, we take code change in segment 17. S17 affects 22 segments, so whenever a test case passes through any of these 23 segments, the relevance ratio of this test case will be incremented by one. Step 5, calculates the relevance ratio at the cluster level where the results are shown in Table 3.

Table 2. Final cluster.

	C(1, 2, 3, 4, 9, 13, 14, 17, 21)	C(5, 6, 10, 11, 12, 15, 16, 20)	c(7, 8)	C(18, 19, 22)
C(1, 2, 3, 4, 9, 13, 14, 17, 21)	0.00	3.78	3.58	3.65
C(5, 6, 10, 11, 12, 15, 16, 20)	3.78	0.00	3.47	3.71
C(7, 8)	3.58	3.47	0.00	3.64
C(18, 19, 21)	3.65	3.71	3.64	0.00

Table 3. Final cluster's relevance ratio.

Cluster	C1	C2	C3
Test cases	T1, T2, T3, T4, T9, T13,T14, T17, T2	T5, T6, T7, T8, T10, T11, T12, T15, T16, T20	T18, T19, T2
Relevance ratio	10.5	11.9	14.3

According to step 5, cluster C3 appears to be the most affected cluster by the change at segment S17. Thus, C3's test cases will be executed first during the regression testing activity. Test cases that belong to C2 will follow the test cases of C3. Finally, cluster C1 shows that it is least affected by this change so test cases of cluster C1 will be the last in the regression testing activity.

After prioritizing clusters in step 6, we need to prioritize the test cases of each cluster in this step, which is calculated based on the test case priority Equation (4). As stated earlier, the test cases of each cluster are prioritized according to the CCg, CCy, and CR for each test case. The test case priority is calculated according to Eq. (4), where the values of $\propto, \beta$, and γ are computed below:

- $\propto = \frac{1}{4}$, where 40 is the total number of methods.
- $\beta = \frac{1}{179}$. There are 14 decision nodes and the sum of all the method code complexity is 109. Referring to Eq. (3), the maximum code complexity is $109 + 5(14) = 179$.
- $\gamma = \frac{1}{200}$, where 200 is the maximum value that can be reached if all the 40 methods' complexity are of high value (equal to 5).

A sample of prioritizing test cases is shown in Table 4. Where the priority of T1 is calculated as follows:

$$\mathrm{T1} = \alpha(\mathrm{CCg}) + \beta(\mathrm{CCy}) + \gamma(\mathrm{RC}) = \frac{1}{40}(25) + \frac{1}{179}(71) + \frac{1}{200}(62) = 1.331648$$

We report in details on two case studies based on the location of change. First, the change location is in segment 31. Second, the change location is in segment 14. In addition to segments 31 and 14, we consider the change in segments 4 and 20. Segments 31, 14, 4, and 20 are chosen as experimental segments because they have an impact on other segments as the flow graph of Fig. 4 shows. This will help in demonstrating the powers of the proposed method. We consider the designated number of clusters to be four. For each case study, an output (T') is generated by the application, where (T') is the prioritized set.

We use inclusiveness and precision metrics to evaluate our proposed method. Inclusiveness measures the degree to which an algorithm chooses test cases that can cause the modified program to produce different output. On the other hand, precision measures the ability of a technique to avoid choosing a test that is not modification-revealing test [16].

Table 4. Sample of prioritizing test cases.

Test case	CCg	Test CCy	Test case RC	Test case priority
T1	25	71	62	1.331648
T2	25	72	68	1.367235
T3	18	46	53	0.971983

If the initial set of test cases (T) contains mr modification-revealing test cases, and the cluster contains smr revealing-modification of these test cases, then the inclusiveness of the algorithm is:

$$\text{ratio (smr/mr), where mr } \neq 0. \tag{5}$$

If the initial set of test cases (T) contains nmr non-modification-revealing test cases, and the cluster omits onmr non-modification-revealing of these test cases, onmr is (total of non-modification-revealing) − (cluster's non-modification-revealing). The precision of the algorithm is:

$$\text{ratio (onmr/nmrmr), where nmr } \neq 0. \tag{6}$$

4.1. *Code change in segment 31*

The code change in S31 affects, as shown in the flow graph in Fig. 4, segments S31, S32, S33, and S40. To show the change-based test case prioritization method results, we have to prove that the listed segments are highly prioritized in the prioritized test case list (T').

The application of the proposed algorithm returns four clusters that are represented in Tables 6–9. The four clusters are prioritized by their index; i.e. clusters 1–4. In these tables, TID represents the test case ID and T refers to whether the test case traverses one of the segments in the above list or not. As shown in the tables, 12 clusters are traversing the modification-revealing segments S31, S32, S33, and S40. For this reason, the modification revealing has a value equal to 12 (mr = 12). On the other hand, 36 ((T) – mr = i.e. $48 - 12 = 36$) test cases are the total non-modification revealing (nmr) test cases; i.e. nmr = 36. Table 5 summarizes the main characteristic of this code change.

Table 5. Characteristics of code change 31.

Number of test cases	Code change	Num affected segments	mr	nmr
48	31	4	12	36

Table 6. Cluster 1 results of S31 change.

TID	T
18	1
34	1
19	1
23	1
29	1
22	1

Note: Inclu. 50%; Prec. 100%.

Table 7. Cluster 2 results of S31 change.

TID	T
5	0
20	1
27	1
10	0
35	0
37	0
11	0
12	0
15	0
16	0
36	0

Note: Inclu. 16.66%; Prec. 75%.

Table 8. Cluster 3 results of S31 change.

TID	T
1	0
4	0
2	0
17	1
21	1
3	0
9	0
24	1
39	0
13	0
33	0
14	0
32	0
6	0
38	0
30	1
7	0
40	0
41	0
28	0
46	0
31	0
47	0
44	0
45	0
8	0
42	0
43	0
48	0

Note: Inclu. 33%; Prec. 27.77%.

Table 9. Cluster 4 results of S31 change.

TID	T
25	0
26	0

Note: Inclu. 0%; Prec. 94.44%.

To calculate the inclusiveness ratio, we need to find for each cluster the smr value:

- For cluster 1, smr = 6; i.e. 6 test cases traverse one or more of the listed segments. Based on equation 4, inclusiveness ratio for cluster 1 is $6/12 = 50\%$.
- For cluster 2, smr = 2; thus, cluster 2 has inclusiveness ratio = $2/12 = 16.66\%$.
- For cluster 3, smr = 4; thus, cluster 3 has inclusiveness ratio = $4/12 = 33.33\%$.
- For cluster 4, smr = 0; i.e. cluster 4 does not include any change-relevant segment at all; thus, cluster 4 has inclusiveness ratio of $0/12 = 0\%$.

To calculate the precision ratio, we need to find for each cluster the non-modification revealing test cases (onmr):

- For cluster 1, onmr = 36–0 = 36. There are zero non-modification revealing test cases; thus, cluster 3 has the precision ration = $36/36 = 100\%$.
- For cluster 2, onmr = 36–9 = 27; thus, cluster 2 has precision ratio = $27/36 = 75\%$.

Table 10. Test cases priority based on change in segment 31.

Test case	CCg	Test CCy	Test case RC	Test case priority
T34	29	83	81	1.5937
T18	28	83	83	1.5787
T19	25	68	75	1.3799
T22	24	76	66	1.3546
T29	24	68	64	1.2999
T23	24	66	59	1.2637
T27	26	73	76	1.4378
T5	25	72	63	1.3422
T11	23	65	68	1.2781
T12	23	64	66	1.2625
T15	24	58	66	1.2540
T10	22	60	68	1.2252
T35	22	59	67	1.2146
T20	22	58	67	1.2090
T36	21	57	61	1.1484
T37	21	57	56	1.1234
T16	20	50	58	1.0693
T39	28	80	74	1.5169
T21	28	82	69	1.5031
T33	26	81	75	1.4775
T9	26	78	71	1.4408

Table 10. (*Continued*)

Test case	CCg	Test CCy	Test case RC	Test case priority
T14	26	76	73	1.4396
T32	26	75	74	1.4390
T17	27	75	68	1.4340
T13	27	75	67	1.4290
T2	25	72	68	1.3672
T1	25	71	62	1.3316
T28	23	65	56	1.2181
T24	22	61	64	1.2108
T6	21	59	60	1.1546
T44	21	59	58	1.1446
T30	21	58	59	1.1440
T45	21	60	55	1.1352
T43	21	58	56	1.1290
T48	21	58	56	1.1290
T7	21	57	56	1.1234
T47	21	57	55	1.1184
T46	21	58	52	1.1090
T42	20	54	58	1.0917
T4	20	56	55	1.0878
T41	20	53	57	1.0811
T40	20	54	54	1.0717
T31	20	54	52	1.0617
T8	19	51	55	1.0349
T38	18	49	55	0.9987
T3	18	46	53	0.9720
T26	11	30	25	0.5676
T25	5	12	13	0.2570

- For cluster 3, onmr= 36–26 =10; i.e. there are 26 non-modification revealing test cases. Based on equation 5, the precision ratio for cluster 3 is $= 10/36 = 27.77\%$.
- For cluster 4, onmr = 36–2 = 34; thus, cluster 4 has precision ratio $= 34/36 = 94.44\%$.

The test cases are prioritized in Table 10 according to the change in segment 31.

4.2. *Code change in segment 14*

The code change in S14 affects, as shown in the flow graph in Fig. 4, segments S14–S16. Tables 12–15 show that the clusters are prioritized differently because our method prioritizes clusters based on the location of the change. This means the cluster that has more relevant test cases to the change is highly prioritized. The modification revealing value equals to 28 (mr = 28). On the other hand, 20 ($(T) - \text{mr} = 48–12 = 36$) test cases are the total non-modification revealing (nmr) test cases; i.e. nmr = 20.

Table 11 summarizes the main characteristics of this code change.

Table 11. Characteristics of code change 14.

Number of test cases	Code change	Num affected segments	mr	nmr
48	14	3	28	20

Table 12. Cluster 1 results of S14 change.

TID	*T*
1	1
4	1
2	1
17	1
21	1
3	0
9	1
24	0
39	1
13	1
33	1
14	1
32	0
6	1
38	0
30	0
7	0
40	0
41	0
28	1
46	1
31	1
47	1
44	1
45	1
8	0
42	0
43	1
48	1

Note: Inclu. 67%; Pre. 50%.

To calculate the inclusiveness ratio, we need to find for each cluster the smr value:

- For cluster 1, smr = 19; i.e. 19 test cases traverse one or more of the listed segments. Based on equation 4, inclusiveness ratio for cluster 1 is $19/28 = 67.85\%$.

Table 13. Cluster 2 results of S14 change.

TID	*T*
18	1
34	1
19	0
23	1
29	1
22	1

Note: Inclu. 17.85%; Prec. 95%.

Table 14. Cluster 3 results of S14 change.

TID	*T*
5	1
20	0
27	0
10	0
35	0
37	0
11	1
12	1
15	0
16	1
36	0

Note: Inclu. −14.28%; Prec. 65%.

Table 15. Cluster 4 results of S14 change.

TID	*T*
25	0
26	0

Note: Inclu. 0%; Prec. 90%.

- For cluster 2, smr = 5; thus, cluster 2 has inclusiveness ratio = 5/28 = 17.85%.
- For cluster 3, smr = 4; thus, cluster 3 has inclusiveness ratio = 4/28 = 14.28%.
- For cluster 4, smr = 0; i.e. cluster 4 does not include any change-relevant segment at all; thus, cluster 4 has inclusiveness ratio = 0/28 which is 0%.

To calculate the precision ratio, we need to find also for each cluster the non-modification revealing test cases (onmr).

- For cluster 1, onmr = 20–10 = 10; i.e. there are 10 non-modification revealing test cases. Based on equation 5, the precision ratio for cluster 1 is 10/20 = 50%.
- For cluster 2, onmr = 20–1 = 19; thus, cluster 2 has precision ratio of 19/20 = 95%.
- For cluster 3, onmr = 20–7 = 13; thus, cluster 3 has precision ratio of 13/20 = 65%.
- For cluster 4, onmr = 20–2 = 18; thus, cluster 4 has precision ratio of 18/20 = 90%.

The test cases are prioritized in Table 16 according to the change in segment 14.

4.3. *Summary of the experimental results*

Tables 17 and 18 show a summary of the experimental results, in addition to the code change in S4 and S20. The tables show that executing the first prioritized cluster will ensure executing test cases more relevant to the change earlier, and it ensures delaying non-relevant test cases to later executions.

Table 16. Test cases priority based on change in segment 14.

Test case	CCg	Test CCy	Test case RC	Test case priority
T39	28	80	74	1.5169
T21	28	82	69	1.5031
T33	26	81	75	1.4775
T9	26	78	71	1.4408
T14	26	76	73	1.4396
T32	26	75	74	1.4390
T17	27	75	68	1.4340
T13	27	75	67	1.4290
T2	25	72	68	1.3672
T1	25	71	62	1.3316
T28	23	65	56	1.2181
T24	22	61	64	1.2108
T6	21	59	60	1.1546
T44	21	59	58	1.1446
T30	21	58	59	1.1440
T45	21	60	55	1.1352
T43	21	58	56	1.1290
T48	21	58	56	1.1290
T7	21	57	56	1.1234
T47	21	57	55	1.1184
T46	21	58	52	1.1090
T42	20	54	58	1.0917
T4	20	56	55	1.0878
T41	20	53	57	1.0811
T40	20	54	54	1.0717
T31	20	54	52	1.0617
T8	19	51	55	1.0349
T38	18	49	55	0.9987
T3	18	46	53	0.9720
T18	28	83	83	1.5787
T19	25	68	75	1.3799
T22	24	76	66	1.3546
T29	24	68	64	1.2999
T23	24	66	59	1.2637
T27	26	73	76	1.4378
T5	25	72	63	1.3422
T11	23	65	68	1.2781
T12	23	64	66	1.2625
T15	24	58	66	1.2540
T10	22	60	68	1.2252
T35	22	59	67	1.2146
T20	22	58	67	1.2090
T36	21	57	61	1.1484
T37	21	57	56	1.1234
T16	20	50	58	1.0693
T34	29	83	81	1.5937
T26	11	30	25	0.5676
T25	5	12	13	0.2570

Table 17. Inclusiveness summary results.

Inclusiveness	C1 (%)	C2 (%)	C3 (%)	C4 (%)
Change in S31	50	16.66	33.33	0
Change in S14	67.85	17.85	14.28	0
Change in S4	20	6.6	60	13.3
Change in S20	6.45	19.35	74.19	0

Table 18. Precision summary results.

Precision	C1 (%)	C2 (%)	C3 (%)	C4 (%)
Change in S31	100	50	27.77	94.44
Change in S14	50	95	65	90
Change in S4	90.90	96.96	39.39	72.72
Change in S20	47.05	100	64.7	100

4.4. *Removing the requirements complexity*

In order to gain a good understanding of the complexities, we prepared an experiment in order to ascertain the relationships of the complexities considered in our work. As we previously discussed, the requirement complexity is one of the main complexities considered in regression testing. It defines the sensitive methods based on the end user requirements, and accordingly it affects the test cases priorities. Thus, to show its affect in prioritizing the test cases, we adjusted the test case priority — TP — to exclude the requirement complexity and produce TP′:

$$\mathrm{TP}' = \alpha(\mathrm{CCg}) + \beta(\mathrm{CCy})$$

The test cases of each cluster are now prioritized according to the CCg and CCy values. The values of $\propto$ and β are computed below:

- $\propto = \frac{1}{40}$, where 40 is the total number of methods.
- $\beta = \frac{1}{179}$. There are 14 decision nodes and the sum of all the method code complexity is 109. Referring to Eq. (3), the maximum code complexity is $109 + 5(14) = 179$.

An example of test case priority calculation is shown below:

$$\mathrm{T1}' = \alpha(\mathrm{CCg}) + \beta(\mathrm{CCy}) = \frac{1}{4}(25) + \frac{1}{179}(71) = 1.0216$$

Table 10 shows the test cases priorities based on the change in segment 31. The priorities of Table 10 were calculated based on the three complexities (TP). However, Table 19 prioritizes the test cases after the change in segment 31 based on the formula TP′. As the table shows, removing the requirement complexity from the formula has a clear impact on the priority of the test cases. T19, T22, T39, T21, T9, T14, T32, T17, T13, T6, T44, T30, T45, T7, T47, T46, T42, T4, T41, T40, and T31 are the test cases that were affected by the change in the test case priority formula. For example, the priority of T32 based on TP is 1.439; however, the priority of T32 decreased to 1.0857 when it was calculated based on TP′. This decrease shifted the

Table 19. Test cases priority based on change in segment 31.

Test case	CCg	Test CCy	Test case priority 2
T34	29	83	1.188687151
T18	28	83	1.163687151
T22	24	76	1.024581006
T19	25	68	1.004888268
T29	24	68	0.979888268
T23	24	66	0.968715084
T27	26	73	1.057821229
T5	25	72	1.027234637
T11	23	65	0.938128492
T12	23	64	0.932541899
T15	24	58	0.924022346
T10	22	60	0.885195531
T35	22	59	0.879608939
T20	22	58	0.874022346
T36	21	57	0.843435754
T37	21	57	0.843435754
T16	20	50	0.779329609
T21	28	82	1.158100559
T39	28	80	1.146927374
T33	26	81	1.102513966
T17	27	75	1.093994413
T13	27	75	1.093994413
T9	26	78	1.08575419
T14	26	76	1.074581006
T32	26	75	1.068994413
T2	25	72	1.027234637
T1	25	71	1.021648045
T28	23	65	0.938128492
T24	22	61	0.890782123
T45	21	60	0.860195531
T6	21	59	0.854608939
T44	21	59	0.854608939
T30	21	58	0.849022346
T43	21	58	0.849022346
T48	21	58	0.849022346
T46	21	58	0.849022346
T7	21	57	0.843435754
T47	21	57	0.843435754
T4	20	56	0.812849162
T42	20	54	0.801675978
T40	20	54	0.801675978
T31	20	54	0.801675978
T41	20	53	0.796089385
T8	19	51	0.759916201
T38	18	49	0.723743017
T3	18	46	0.70698324
T26	11	30	0.442597765
T25	5	12	0.192039106

priorities of the test cases and did not consider T32 as a high priority test case. Even though, T32 has a high requirement complexity and the methods that are included in this test cases are important to the users and must be error free, all the three complexities are important in identifying the priorities of the test cases and eliminating any of this complexity will drastically affect the results of the regression testing.

4.5. *Comparison of results with previous works*

In this section, we performed a hypothetical comparison between our approach and previously suggested approaches. Sherrif *et al.* [19] discussed in their paper the effects of a change in the code of a tested system; a change in any method can have consequences on the whole methods of the system and not just the related segments. In their paper, they proposed a methodology that can determine the effects of the change which allows them to prioritize the test cases based on the results of the methodology. The methodology for measuring the affected test cases is based on the historical data. That is, the methodology generates clusters of files that historically tend to change together. Then, their approach combines the clusters with the test case information which yield to a matrix that can be multiplied by a vector representing the system modification. The approach they represented was tested on a software product at IBM.

The authors used four metrics in order to compare their results; these metrics are inclusiveness, precision, efficiency, and generality. In this section, we compare the inclusiveness and precision results of their approach with our results. The inclusiveness measures how much a technique is affective by finding the affected test cases. The authors used the historical data in order to identify the effect of the change. This showed important results when the change that is occurring was already included in the history. However, if a new system modification is presented, then new methods might be affected by a change that was never affected before. In this case, their methodology cannot trigger the test cases that can be affected by the change; and thus, the prioritization will not be useful. For this reason, the inclusiveness values of our approach overcome their inclusiveness value where the authors indicated this by saying "Our regression test prioritization is not considered safe because of our use of historical data in our impact analysis aspect in the technique". On the other hand, our approach is capable of finding the effected test cases regardless if it a new change or not. The case studied in our approach proved that running the first set of the prioritized test cases is capable of triggering many faults that result from the change. The methodology proposed in [19] shows a good precision value, and this is due to the use of historical data. This is indicated by the authors when saying "If new features are not being added to a system and new modifications follow historical patterns, then the precision of our technique could be high." On average the precision value of our approach is 73% which is slightly high. However, there should be a tradeoff between false positive and false negative. It is better to have a safe methodology that

causes extra test cases to be executed than missing a test case that can reveal some faults in the system. Our method is considered to be safer since is does not miss test cases that can reveal faults.

Moreover, we compared our approach to that presented by Beszedes *et al.* in [20]. Beszedes *et al.* tackled, in their paper, the problem of regression testing. In many cases, regression testing results in a test suite that is too large to extend. In order to minimize the cost of regression testing, the authors prioritize the test cases based on code coverage. The approach was tested on the open source web browser engine — the WebKit system. The authors conducted experiments that proved that applying prioritization is very beneficial since it limits the number of test cases that needs to be rerun. However, the authors used only one metric to evaluate their work — inclusiveness. This, we believe, is a major drawback, since the strength of the work is only based on one metric. Although the inclusiveness metric is one of the most important metrics that enables the researches to specify how many failing test cases are included in the cluster, it is also important to consider other metrics such as precision. The results of their experiments showed good inclusiveness results in most of the cases; thus achieving the minimization in the selection of regression tests. However, in order to prove their approach, they had to modify different parts of the WebKit system. Moreover, the approach did not show good inclusiveness in some of the cases. This was clear when the authors stated "due to the limitations required by practical constraints, the inclusiveness is not perfect". Our approach prioritizes test cases based on the change that occurred taking into consideration more than one complexity (code complexity, user requirement complexity, and code coverage) which enabled us to know more about the effect of change on the methods; whereas, the approach presented in [20] used only one complexity (code coverage) to specify the effect of the change. Our proposed approach was tested on several code changes and the inclusiveness was relatively high in all the experiments conducted.

5. Conclusion

When software is deployed in sensitive environments where errors can lead to dramatic changes, it is crucial to conduct effective software testing. This testing is also required when software is updated. However, software testing is an expensive step especially when we have to test large systems again after a small modification to this system. Then, testing might cost more than the change itself. Regression testing is a software testing method that generates a subset of test cases that should be rerun to provide confidence that the software is bug-free. Although regression testing minimizes the cost, when the system under question is large, regression testing cannot minimize the cost alone. In this paper, we proposed an approach that can address this problem by reducing the cost of software testing. We proposed a new change-based test case prioritization method that is based on the clustering approach. In addition, we introduced a new prioritization metric — requirement complexity. Initially, we clustered similar test cases by using the Agglomerative hierarchical method. Next,

we prioritized the clusters based on their relevance to the change location. Then, we prioritized test cases within each cluster based on three metrics: CCy, CCg, and RC.

To evaluate our method, we carried out an experimental study over four case studies based on the LIP system. First, we clustered the 48 test cases into four clusters. Then, for each case study, we inserted a code change in a specific segment (location) in order to prioritize clusters based on the change that occurred. Each change revealed actual affected segments. The experiments show that the first prioritized cluster has more advantage over the other clusters by means of inclusiveness and precision metrics. The second case study showed that executing test cases of the first cluster contains 67.85% of the affected test cases. This means that the test cases are prioritized in a way that allows the more important test cases to be executed first. In order to show the strength of our approach, we performed a comparison with other work. The comparison shows that our approach is better in detecting the test cases that are affected by the change where our inclusiveness values are relatively high.

As for future work, we plan to extend the validation of our results to large systems. We also plan to explore the performance of our methodology at different levels of granularity.

Appendix A

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.Data.SqlClient;
using Mommo.Data;

namespace TestCaseSOMClustering
{
  public partial class Form1 : Form
  {
    float[,] TestCasesDistance;
    float[,] ClusterDistance;
    int[ ] TraveredMethods;
    int[ ] TestCasesRelevanceRatio;
    int[ ] TestCasesCodeCoverage;
    int[ ] TestCasesCodeComplexity;
    int[ ] TestCasesRequirementComplexity;
    ArrayList DeletedClusters;
```

```
testcase[ ] PrioritizedTestCases;
int numberTestCases = 48;
int numberMethods = 40;
int numberOfOutClusters = 4;
int numberCurrentCluster;
Cluster[ ] Clusterlist;
string ConnectionString;
public Form1()
{
  ConnectionString = "Data Source=LOUMA-PC;Initial Catalog=test;User
  ID=sa;Password=sa;";
  InitializeComponent();
  numberCurrentCluster = numberTestCases;
  DeletedClusters = new ArrayList();
  PrioritizedTestCases = new testcase[numberTestCases];
}
// First Step
private void button1_Click(object sender, EventArgs e)
{
  CreateTestCasesDistanceArray();
  InitilizeClusters();
  InitializeClusterDistances();
  DisplayClusterDistances();
}
// Second Step
private void btnInitializeClusters_Click(object sender, EventArgs e)
{}
// Third step
private void btnMergeClosestClusters_Click(object sender, EventArgs e)
{
  txtPairs.Clear();
  while(numberCurrentCluster > numberOfOutClusters)
    MergeClosestClusters();
  CalculateClusterRelevanceRatio();
  int index = 0;
  // sort clusters first
  foreach (Cluster c in Clusterlist)
  {
    if (c.status == "EMPTY")
      continue;
    for (int i = 0; i < c.TestCases.Count; i++)
    {
      PrioritizedTestCases[index + i] = new testcase();
```

```
            PrioritizedTestCases[index + i].testCaseID = (int)c.TestCases[i];
            PrioritizedTestCases[index + i].codeComplexity = TestCasesCode-
            Complexity[(int)c.TestCases[i]];
            PrioritizedTestCases[index + i].clusterID = index;
          }
          ++index;
        }
        DisplayClusterDistances();
      }
void CreateTestCasesDistanceArray()
{
   TestCasesDistance = new float[numberTestCases, numberTestCases];
   try
   {
     SqlConnection Conn = new SqlConnection(ConnectionString);
     Conn.Open();
     SqlCommand command;

     for (int i = 0; i < numberTestCases; i++) //number of testcases
     {
       for (int j = 0; j < numberTestCases; j++)
       {
         if (j < i)
           TestCasesDistance[i, j] = TestCasesDistance[j, i];
         else
         {
           command = new SqlCommand("dbo.GetCasesDistance", Conn);
           command.CommandType = CommandType.StoredProcedure
           SqlParameter param = command.Parameters.Add("@testCaseID1",
           SqlDbType.Int);
           param.Direction = ParameterDirection.Input;
           param.Value = i;
           param = command.Parameters.Add("@TestCaseID2", SqlDbType.Int);
           param.Direction = ParameterDirection.Input;
           param.Value = j;
           param = command.Parameters.Add("@Distance", SqlDbType.Float);
           param.Direction = ParameterDirection.Output;
           command.ExecuteNonQuery();
           TestCasesDistance[i, j] = float.Parse(command.Parameters["@Distance"].
           Value.ToString());
           int t = 0;
           if (j == 95)
             t++;
```

```
            if (TestCasesDistance[i, j] == 0 && i != j)
            {
              MessageBox.Show(i.ToString() + "and" + j.ToString() + "have same
              path");
              DeleteTestCase(j);
              numberTestCases–;
            }
          }
        }
    }
}
catch (Exception exc)
    {
      MessageBox.Show(exc.Message);
    }
}
void DeleteTestCase(int testCaseID)
{
    try
    {
      SqlConnection Conn = new SqlConnection(ConnectionString);
      Conn.Open();
      SqlCommand command;
      command = new SqlCommand("dbo.DeleteTestCase", Conn);
      command.CommandType = CommandType.StoredProcedure;
      SqlParameter param = command.Parameters.Add("@testCaseID", SqlDbType.
      Int);
      param.Direction = ParameterDirection.Input;
      param.Value = testCaseID;
      command.ExecuteNonQuery();
    }
catch (Exception exc)
    {
      MessageBox.Show(exc.Message);
    }
}

void InitilizeClusters()
{
    // create a cluster for each test case
    Clusterlist = new Cluster[numberTestCases];
    for (int i = 0; i < numberTestCases; i++)
      Clusterlist[i] = new Cluster(i, i);
```

```
}
void InitializeClusterDistances()
{
   ClusterDistance = new float[numberTestCases, numberTestCases];
   ClusterDistance = TestCasesDistance;
}
void GetMinClusterDistance(ref int n, ref int k)
{
   float min = 1000;
   for (int i = 0; i < numberTestCases; i++) //number of test cases
   {
     if (Clusterlist[i].status == ″EMPTY″)
        continue;
     for (int j = i + 1; j < numberTestCases; j++)
     {
        if (Clusterlist[j].status == ″EMPTY″) // dont include the empty clusters
           continue;
        if (ClusterDistance[i, j] < min) // dont include the deleted clusters
        {
           min = ClusterDistance[i, j];
           n = i;
           k = j;
        }
     }
   }
}
float GetMinDistance()
{
   float minDistance = 1000;
   for (int i = 0; i < numberTestCases; i++) //number of test cases
   {
     if (Clusterlist[i].status == ″EMPTY″)
        continue;
     for (int j = i + 1; j < numberTestCases; j++)
     {
        if (Clusterlist[j].status == ″EMPTY″) // dont include the empty clusters
           continue;
        if (ClusterDistance[i, j] < minDistance) // dont include the deleted clusters
           minDistance = ClusterDistance[i, j];
     }
   }
   return minDistance;
```

```
}
bool ClustedIsRemoved(int ClusterID)
{
   for (int i = 0; i < DeletedClusters.Count; i++)
   {
     if (ClusterID == i)
        return true;
   }
   return false;
}
float CalculateDistance(Cluster src, Cluster dest)
{
   float distance = 0;
   int count = 0;
   if (src.ClusterID == dest.ClusterID)
      return 0;
   for (int i = 0; i < src.TestCases.Count; i++)
      for (int j = 0; j < dest.TestCases.Count; j++)
   {
      count++;
      distance += TestCasesDistance[(int)src.TestCases[i], (int)dest.TestCases[j]];
// test case ID is the index
   }
   return distance/count; // average
}
void MergeClosestClusters()
{
   float MinDistance = GetMinDistance();
   for (int i = 0; i < numberTestCases; i++) //number of testcases
   {
      if (Clusterlist[i].status == ″EMPTY″)
         continue;
      for (int j = i + 1; j < numberTestCases; j++)
      {
         if (Clusterlist[j].status == ″EMPTY″) // dont include the empty clusters
            continue;
         if (ClusterDistance[i, j] == MinDistance) // dont include the deleted clusters
         {
            Clusterlist[i].AddToCluster(Clusterlist[j]); // merge cluster j with cluster i
```

```
            Clusterlist[j].EmptyCluster();
            DeletedClusters.Add(j);
            ReclaculateClusterDistance(i); // Cluster j is merged to cluster i
            numberCurrentCluster–;
          }
        }
    }
}
void ReclaculateClusterDistance(int clusterID)
{
   for (int j = 0; j < numberTestCases; j++)
      if (Clusterlist[j].status != ″EMPTY″)
      {
         ClusterDistance[clusterID, j] = CalculateDistance(Clusterlist[clusterID],
 Clusterlist[j]);
         ClusterDistance[j, clusterID] = ClusterDistance[clusterID, j];
      }
}
void DisplayClusterDistances()
{
   string[ ] colnames = new string[numberTestCases];
   for (int c = 0; c < numberTestCases; c++)
   {
      colnames[c] = ″C(″;
      for (int i = 0; i < Clusterlist[c].TestCases.Count; i++) // add the test cases to
      the cluster
       colnames[c] + = Clusterlist[c].TestCases[i].ToString() + ″ ″;
       colnames[c] = colnames[c].Remove(colnames[c].Length - 1);
       colnames[c] + = ″)″;
   }
dGridDistances.DataSource = new Mommo.Data.ArrayDataView(ClusterDistance,
colnames);
txtPairs.Clear();
foreach (Cluster c in Clusterlist)
{
   if (c.status != ″EMPTY″)
      for (int j = 0; j < c.TestCases.Count; j++)
         txtPairs.Text + = c.TestCases[j] + ″, ″ ;
   else
   {
      for (int i = 0; i < numberTestCases; i++)
```

```
      {
        dGridDistances.Rows[c.ClusterID].Cells[i].Style.BackColor = System.Draw-
        ing.Color.DarkGray;
        dGridDistances.Rows[i].Cells[c.ClusterID].Style.BackColor = System.Draw-
        ing.Color.DarkGray;
        Label label = new Label();
      dGridDistances.Rows[c.ClusterID].Cells[i].Style.Font = new Font(label.Font,
      FontStyle.Strikeout) ;
      dGridDistances.Rows[i].Cells[c.ClusterID].Style.Font = new Font(label.Font,
      FontStyle.Strikeout);
      }
      continue;
    }

    txtPairs.Text + = ":" + c.RelevanceRatio + ";";
    txtPairs.Text + = System.Environment.NewLine;
   }
}

private void btnTraversedMethods_Click(object sender, EventArgs e)
{
   String s = txtMethodID.Text;
   if (txtMethodID.Text == "")
   {
      MessageBox.Show("please Enter Valid SegmentID where the change hapened");
      return;
   }
   TraveredMethods = new int[numberMethods];
   for (int i = 0; i < TraveredMethods.Length; i++)
      TraveredMethods[i] = 0; // non of the methods travered yet

   ResetMethodRelevance();
   UpdateDirectDependentMethod(int.Parse(txtMethodID.Text));

   // Display Dependent Method
   for (int i = 0; i < TraveredMethods.Length; i++)
      if(TraveredMethods[i] == 1)
         txtPairs.Text + = (i + 1).ToString() + System.Environment.NewLine; //
means non of the methods travered yet

}
void ResetMethodRelevance()
{
   SqlConnection Conn = new SqlConnection(ConnectionString);
   Conn.Open();
```

```
   SqlCommand command;
   command = new SqlCommand(″dbo.ResetMethodRelevance″, Conn);
   command.CommandType = CommandType.StoredProcedure;
   command.ExecuteNonQuery();
   Conn.Close();
}
void UpdateDirectDependentMethod(int MethodID)
{
   TraveredMethods[MethodID - 1] = 1;
   SqlConnection Conn = new SqlConnection(ConnectionString);
   Conn.Open();
   SqlCommand command;
   command = new SqlCommand(″dbo.GetDependentMethods″, Conn);
   command.CommandType = CommandType.StoredProcedure;
   SqlParameter param = command.Parameters.Add(″@SrcID″, SqlDbType.Int);
   param.Direction = ParameterDirection.Input;
   param.Value = MethodID;

   SqlDataAdapter adapter = new SqlDataAdapter(command);
   DataSet Ds = new DataSet();
   adapter.Fill(Ds, ″Methods″);
   Conn.Close();
   foreach (DataRow row in Ds.Tables[″Methods″].Rows)
   {
      if (TraveredMethods[int.Parse(row[″MethodDestID″].ToString()) - 1] == 1)
         continue;
      TraveredMethods[int.Parse(row[″MethodDestID″].ToString()) - 1] = 1; //
       methods are 1 based
      UpdateDirectDependentMethod(int.Parse(row[″MethodDestID″].ToString()));
   }
}
void GetTestCaseRelevanceRatio()
{
   TestCasesRelevanceRatio = new int[numberTestCases];
   TestCasesCodeCoverage = new int[numberTestCases]; ;
   TestCasesCodeComplexity = new int[numberTestCases]; ;
   TestCasesRequirementComplexity = new int[numberTestCases];
   SqlConnection Conn = new SqlConnection(ConnectionString);
   Conn.Open();
   SqlCommand command;
   command = new SqlCommand(″dbo.GetTestCaseRelevanceRatio″, Conn);
   command.CommandType = CommandType.StoredProcedure;
```

```
  SqlDataAdapter adapter = new SqlDataAdapter(command);
  DataSet Ds = new DataSet();
  adapter.Fill(Ds, "TestCaseRatio");
  Conn.Close();

  for (int i = 0; i < numberTestCases; i++)

  {
    TestCasesRelevanceRatio[i] = int.Parse(Ds.Tables["TestCaseRatio"].Rows[i]["
    Relevance"].ToString());
    TestCasesCodeCoverage[i] = int.Parse(Ds.Tables["TestCaseRatio"].Rows[i]["
    NCoveredMethods"].ToString());
    TestCasesCodeComplexity[i] = int.Parse(Ds.Tables["TestCaseRatio"].Rows[i]["
    CodeComplexity"].ToString());
    TestCasesRequirementComplexity[i] = int.Parse(Ds.Tables["TestCaseRatio"].
    Rows[i]["RequirementComplexity"].ToString());
  }
}
void CalculateClusterRelevanceRatio()
{
  GetTestCaseRelevanceRatio();
  int Ratio = 0;

  foreach (Cluster c in Clusterlist)
  {
    if (c.status == "EMPTY")
      continue;
  Ratio = 0;
  for (int i = 0; i < c.TestCases.Count; i++)
  {
    Ratio + = TestCasesRelevanceRatio[int.Parse(c.TestCases[i].ToString())];
  }
  c.RelevanceRatio = (float)Ratio / c.TestCases.Count;
}
}
private void button1_Click_1(object sender, EventArgs e)
{
  Form2 frm = new Form2();
  frm.Show();
    }
   }
}
```

References

1. R. Carlson, H. Do and A. Denton, A clustering approach to improving test case prioritization: An industrial case study, in *IEEE Int. Conf. Software Maintenance* **27** (2011) 382–391.
2. G. Rothermel, U. Roland, Untch, C. Chu and M. Harrold, Prioritizing test cases for regression testing, *IEEE Trans. Software Eng.* **27**(10) (2001) 929–948.
3. A. Lenz, A. Pozo and S. Vergilio, An approach for clustering test data, *Latin American Test Workshop* **12** (2011) 1–6.
4. C. Simons and E. Parasio, Regression test cases prioritization using failure pursuit sampling, in *Int. Conf. Intelligent Systems Design and Applications*, Vol. 10, 2010, pp. 932–928.
5. H. Xian, A test case design algorithm based on priority techniques, in *Int. Conf. Management of e-Commerce and e-Government*, Vol. 5, 2011, pp. 75–62.
6. M. Kayes, Test case prioritization for regression testing based on fault dependency, *Int. Conf. Electronics Computer Technology* **5**(3) (2011) 48–52.
7. B. Zhang, W. K. Chan and T. Tse, Adaptive random test case prioritization, in *IEEE/ACM Int. Conf. Automated Software Engineering*, 2009, pp. 5584–5593.
8. K. Walcott, M. Soffa, G. Kapfhammer and R. Roos, TimeAware test suite prioritization, in *Int. Symp. Software Testing and Analysis*, 2006, pp. 1–12.
9. L. Zhang, S. Hou, C. Guo, R. Xie and H. Mei, Time-aware test-case prioritization using integer linear programming, in *Int. Symp. Software Testing and Analysis*, 2009, pp. 213–224.
10. J. Kim and A. Porter, A history-based test prioritization technique for regression testing in resource constrained environments, Technical report, Department of Computer Science, University of Maryland, College Park, Maryland, USA, 2002.
11. S. Mirarab, S. Akhlaghi and L. Tahvildari, Size-constrained regression test case selection using multicriteria optimization, *IEEE Trans. Software Eng.* **38**(4) (2012) 936–956.
12. A. Pravin and S. Srinivasan, Effective test case selection and prioritization in regression testing, *J. Comput. Sci.* **9**(5) (2013) 654–659.
13. D. Leon and A. Podgurski, A comparison of coverage-based and distribution-based techniques for filtering and prioritizing test cases, in *Int. Symp. Software Reliability Engineering*, Vol. 14, 2003, pp. 442–453.
14. A. Gonzalez, E. Piel and H. Gross, Prioritizing tests for software fault localization, in *Int. Conf. Quality Software*, Vol. 10, 2010, pp. 42–51.
15. D. Jeffrey and A. Gupta, Test case prioritization using relevant slices, in *Ann. Int. Computer Software Applications Conference*, Vol. 30, 2006, pp. 411–420.
16. S. Elbaum, A. Malishevsky and G. Rothermel, Test case prioritization: An empirical study, *IEEE Trans. Software Eng.* **28**(2) (2002) 179–188.
17. G. Rothermel and M. Harrold, Analyzing regression test selection techniques, *IEEE Trans. Software Eng.* **22**(8) (1996) 529–551.
18. A. Singla and Karambir, Comparative analysis & evaluation of Euclidean distance function & Manhattan distance function using k-means algorithm, *Int. J. Adv. Res. Comput. Sci. Software Eng.* **2** (2012) 298–300.
19. M. Sherrif, M. Lake and L. Williams, Prioritization of regression test using singular value decomposition with empirical change records, in *Proc. of 18th IEEE Int. Symp. on Software Reliability*, 2007, pp. 81–90.
20. A. Beszedes, T. Gergely, L. Schrettner, J. Jasz, L. Lango and T. Gyimothy, Code coverage-based regression test selection and prioritization in webkit, in *IEEE Int. Conf. Software Maintenance*, 2012, pp. 81–90.